Shims and dispatch

TBK SPEC 07:2026, Version 1.0

The Tebako Project

Unrestricted
November 01, 2026
TBK SPEC 07:2026, Version 1.0
Tebako
© 2026 The Tebako Project. All rights reserved

Ribose

Contents

Foreword 4

0. v1 concrete choices (normative where the sections above were open) 4

1. The model 5

2. The dispatch chain (per invocation of `~/.tebako/shims/<tool>`) 6

3. Shell integration (no per-shell magic) 8

4. Configuration 9

5. Distribution forms (both produced by `tebako press`) 10

6. Retirement gate 10

7. Errors (named) 10

8. Native exec from inside an image (the whole-chain model, locked) 11

8a. The virtual cwd (tebako#734) 14

9. Environment composition (locked 2026-07-27) 15

9.1 The interp_env chain (IMPLEMENTED 2026-10-07 — tebako#559; spec 03 §2.7) 15

10. The runtime-own command surface (DRAFT — the user-direct grammar) 16

© 2026 The Tebako Project

All rights reserved. Unless otherwise specified, no part of this publication may be reproduced or utilized otherwise in any form or by any means, electronic or mechanical, including photocopying, or posting on the internet or an intranet, without prior written permission. Permission can be requested from the address below.

The Tebako Project

Suite 1, 8/F, New Henry House
10 Ice House Street
Central
Hong Kong

copyright@ribose.com
www.ribose.com

© 2026 The Tebako Project

Foreword

Normative specification of executable registration and version management. Status: PARTIAL — the dispatcher and version manager (§2–§4) ship as `crates/tebako-shim` in tebako-rs (roadmap 08; retires `mnenv`). Remote registry fetch ships with `tebako add-registry | install` (roadmap 28.1 — the CLI resolves every registry form of spec 04 §2), and the dispatch-time registry cache ships with roadmap 33 (the registry-default chain link resolves every registry form through tebako-resolve behind `~/.tebako/registries/<sha>.yaml` — 24 h TTL, `tebako update-registries`, `TEBAKO_OFFLINE` = cache-or-named-error; `tebako add-registry` primes the cache with the bytes it fetched; a STALE cache + a failed refresh serves the stale bytes LOUD — stderr + journal — rather than failing, spec 05 §4's stale-serve, roadmap 86). The 2026-09-05 routing amendment (qualified pins, per-payload disable, `tebako-shim use`) SHIPS with this spec's PR-A — the status lines below reflect it. Still PLANNED: jail application (spec 08), and a runtime registry (v1 downloads resolve through the `runtimes:` preference in `config.yaml` as the exact ref).

0. v1 concrete choices (normative where the sections above were open)

- Project pin file `.tebako-tools.yaml` is a FLAT YAML mapping of command name → pin; a nearer file that does not pin the command does not shadow a farther one that does.
- The chain-value grammar (2026-09-05 amendment): every link of the version chain — `TEBAKO_<TOOL>_VERSION`, `.tebako-tools.yaml`, `config.yaml defaults:` — carries `[payload@]version` (split on the FIRST `@`; `tpkg::toolpin::ToolPin` is the ONE parser — spec 00 invariant 10). `payload@version` makes the named payload THE provider (§2 step 0.5); bare `version` keeps today's scan. A payload-only pin

© 2026 The Tebako Project

(payload@) does NOT exist — route-out rides disable, never an incomplete link. An unparseable chain value is the NAMED grammar error (EX_TEBAKO_MANIFEST, naming the link and value), never a silent skip.

- **~/.tebako/config.yaml keys:** defaults: (command → [payload@]version; tebako#667: a KEY of the form <registry>/<payload> is the payload-level pin — it covers every entrypoint the payload declares, and a command-level pin beats it for its own command, §4), registries: (spec 04 refs), runtimes: (engine → {version, tebako, source?} runtime preference; source: pins the engine's download base — spec 05 §2's per-engine chain, SHIPPED in the shim's dispatch-time fetch, tebako#567). The shim never writes this file — except through tebako-shim use, the explicit verb (tmp + rename, the same discipline as add-registry; a structural edit that preserves keys, not comments).
- Disabled state is shim-managed state, not authored config: ~/.tebako/shims/.disabled.yaml (command → selector list). The selector grammar (2026-09-05 amendment — tpkg::toolpin:: DisableSelector is the ONE parser): all gates every claim of the command; version gates that version of ANY claim; payload@all gates the named payload's claim; payload@version gates exactly that (payload, version) claim. An unknown selector string in the file is a NAMED parse error at load, never silently ignored (invariant 9).
- Installed payload record (the dispatcher-visible manifest mirror, spec 03 §4 tier 3 rationale): payloads/<name>/<version>.tfs, <version>.tfs.sha256, <version>.manifest.yaml.
- Zero-runtime dispatch (spec 03 §2.2): the payload image itself is the program — the self-launching-image contract; dependency mounts still ride the ABI v1 argv shape minus the runtime prefix.
- Registry payload mounts are bare images: slot 0 (whole image), mount point / of the jail namespace.

1. The model

Tebako manages shims for **every executable every installed payload provides** (spec 03 entrypoints). One payload may carry MULTIPLE executables — each is dispatchable; the ACTIVE subset (spec 03 §2.2's per-entrypoint active, default true) is PATH-registered at install, and an inactive-but-declared command links on demand (tebako shim enable <name>). A payload's kind: runtime edge (spec 30) adds one more source of names: the edge's expose: list registers one shim per exposed depended-runtime entry under the same active-flag and enable-on-demand rules — dispatch of such a shim IS spec 30 §2's spawned dispatch (argv0 → the payload's version chain → the depended runtime's resolution), never a direct store-path exec. Four artifacts, four jobs:

- payload — a signed .tfs image (versioned, runtime-independent).

© 2026 The Tebako Project

- **runtime** — a signed runtime payload (versioned, cached, machine-shared).
- **registry** — a developer-hosted `tpkg-registry.yaml` (spec 04 §2).
- **dispatcher** (`tebako-shim`, a tiny static Rust binary in tebako-rs) — the thing on PATH that picks version + runtime per invocation and hands off.

Provider routing (2026-09-05 amendment): two installed payloads may provide the SAME command (forks, alternative distributions, a suite and a standalone tool). Provider resolution consults two LOCAL surfaces — the version chain's payload-qualified pins (§2 step 0.5) and the disabled-state file's payload-qualified selectors — never the registry: a registry's `default:` stays payload-keyed, and a registry NEVER routes commands (routing is local state by design; a future registry `default_payload:` cold-start hint is a separate extension point, not this spec). The version chain itself is a DYNAMIC per-tool family (`TEBAKO_<TOOL>_VERSION` — an unbounded name set) with its own four-link chain (§2.1); it is NOT a citizen of spec 23 §14's settings registry (§14 owns FIXED-NAME press/machine knobs) — the two surfaces never drift into each other.

argv0 integrity (locked 2026-08-29): command selection rides argv0 — the invoked name must reach the process as its own argv[0]. Symlinks, hardlinks, and byte copies preserve it. Wrapper generators that spawn the target under the TARGET's own path (chocolatey's shimgen — `Install-BinFile`; a shell wrapper that `exec`s the real path) silently defeat selection: every invocation then falls back to the primary entry (`entries[0]`) and the other declared commands are unreachable. A distributor exposes a multi-command package as real links or copies, one per declared entrypoint name — never as re-exec wrappers.

2. The dispatch chain (per invocation of `~/.tebako/shims/<tool>`)

0 **argv0 is the selector.** One tebako-shim binary, linked per command name; it maps name → entrypoint in the payload's manifest. **Multi-command suites:** one package with N entrypoints installs N shims — each dispatches to its own image AND ITS OWN runtime requirement; two commands in one package may run different runtime versions simultaneously.

0.5 **PROVIDER ROUTING (2026-09-05 amendment):** the chain VALUE grammar is `[payload@]version` (`tpkg::toolpin` is the SSOT parser). A payload-qualified value makes the named payload THE provider — it must be installed and declare or expose the command, else the named NotAProvider error. The provider scan skips claims disabled at `<payload>@all` or `<payload>@version`, so disabling all-but-one claims routes without a pin. Unqualified values fall through to today's scan (payload-of-command-name fast path → suite scan → expose scan). A registry never routes commands: routing is local state.

© 2026 The Tebako Project

1. **Payload VERSION resolution** (first match wins): `TEBAKO_<TOOL>_VERSION` env → nearest `.tebako-tools.yaml` walking up from cwd (per-project pinning) → user default (`tebako-shim use <tool> <pin>`) → registry's `default`.
2. **RUNTIME resolution:** the entrypoint's `runtime_requirement` → newest COMPATIBLE cached runtime (no download) → else download newest compatible (spec 05 §5). **Swapping runtimes needs no payload change** — the payload is immutable; only the dispatch-time choice changes (`tebako use --runtime ruby@3.4.2`, or a per-project pin).
3. **EXTENSION-SLICE scan (additive — schema_minor 10, spec 03 §2.8):** when the resolved root payload declares `extension_points:`, the dispatcher scans the STORE's payload manifest mirrors (`payloads/*/*.manifest.yaml`, spec 05 §3) for slices whose `augments[].payload` names the root — store-local, never a network fetch. Per candidate: bind `extension_point` → mount `<point.mount>/<slice.name>` (an unknown point name = loud skip); check the slice's `constraint` against the RESOLVED base version and its language edge against the RESOLVED runtime (engine + abi line). A slice MAY carry several `augments` edges naming the root — the suffixed-line family (spec 03 §2.8) ships one exact-pin edge per family row; the candidate attaches when ANY such edge matches. Newest compatible version per slice name attaches (the store layout makes same-version duplicates impossible); an auto-discovered mismatch = loud skip + journal (`event=slice-skip slice=… reason=…`); a successful attach journals `event=slice-augment slice=… base=… mount=…`. Compatible slices append `--tebako-image <slice-path>:-:<mount>` triples AFTER the app's (spec 17 multi-mount — nested under the app's `/` mount, longest-prefix dispatch; a mount failure is the driver's all-or-nothing teardown). Config-pinned slices (§4) override auto-discovery per slice name, fetch on miss (parity with the runtime fetch; `TEBAKO_OFFLINE=1` = cache-or-error), and a pinned mismatch is the named SliceIncompatible error, never a skip. When base and attaching slice both carry `provides.gems`, a same-name gem pair journals `event=slice-overlap slice=… gem=…` — informational only (gem homes compose by requirement-based activation; the fix is a slice rebuild against the newer base).
4. **Hand-off:** mount payload + ZERO OR MORE runtime payloads (native entrypoints need none — spec 03) + declared dependency mounts (spec 03 §2.3), apply the jail view (spec 08), exec the entrypoint. Signed payloads are verified at install time, not per run. When the payload declares `kind: runtime` edges (spec 30), the dispatcher resolves each edge NOW (cache or download per spec 05 §5) and exports the pins as `TEBAKO_SPAWN_LOCK` (spec 17 §2) — the driver's spawn-time picks are locked to the dispatch-time resolution and a store GC cannot re-pick mid-process.

© 2026 The Tebako Project

3. Shell integration (no per-shell magic)

- ONE directory on PATH: `~/.tebako/shims`. One-time setup; `tebako shim install-shell [--shell bash|zsh|fish|csh]` inserts a managed BEGIN/END block into the right startup file (`.profile`/`.bash_profile`/`.bashrc`/`.zshrc`/`.cshrc`) prepending the shim dir; idempotent; `uninstall-shell` removes exactly its block.
- Shim links are always an explicit act: registry installs register the payload's `provides` (that IS the install's point); a local-package install (`tebako install <path>`) links only with the explicit `--shims`; a run — of any package, ever — links nothing. A one-off act never claims PATH names.
- NO eval-init hook for switching: the dispatcher reads the project file itself (the mise model, not the rbenv `eval "$(... init -)"` model).
- `tebako-shim use / disable / enable / list / which / doctor` manage shims (link/remove/inspect/diagnose); enable/disable specific versions. The verbs spell identically through the CLI: `tebako shim <verb> ...` ≡ `tebako-shim <verb> ...` (tebako-cli calls the shim crate as a library — never a shell-out).
  - **`use <tool> <pin>`** writes the user-default link (`config.yaml defaults:` — the explicit authored-config write verb beside `add-registry`, tmp + rename, keys preserved); the pin is the `[payload@]version` grammar of §0. `use --clear <tool>` removes exactly that key; `use --runtime <engine>@<langver>[:<tebako>]` writes `runtimes:`.
  - **`enable|disable <tool>[@<version>] [--of <payload>]`** edit the disabled-state selectors of §0: bare → `all`; `@<version>` → `version`; `--of <payload>` alone → `payload@all`; with `@<version>` → `payload@version`.
  - **`which <tool>`** names the provider payload and provider kind (own / exposed / pinned) beside the version and its source.
  - **`list [--json]`** shows per command the provider payload, provider kind, resolved version + source, and disabled marks; `--json` emits one `"info_schema": 1` document (mirroring `tebako cache list --json`).
  - **`doctor`** additionally reports: commands with MORE THAN ONE enabled claim (collisions), dangling pins (project-file/config pins naming an uninstalled payload or version), and disabled-but-pinned conflicts. Doctor stays diagnose-only: findings print, the exit code is unchanged (0 / 1 with problems).

    Enable links on demand (locked 2026-08-29): `enable <tool>` whose command is declared by an installed payload but was never linked (spec 03 §2.2's `active: false` default) materializes the shim link then — the dispatcher links ITSELF (argv0 integrity above); enabling an undeclared command is the suite scan's named error, never a dangling link. `disable` gates per version (`.disabled.yaml`) without removing the link.

© 2026 The Tebako Project

4. Configuration

- User config: `~/.tebako/config.yaml` (YAML — the locked convention; supersedes the earlier `config.json` note). Contents: defaults, registries, runtime preferences. (IMPLEMENTED 2026-10-07 — tebako#559: a per-tool `env:` map, store-config schema minor 3 — the user-config layer of the §9.1 interp_env chain, set by hand or by `tebako-shim env`; never written implicitly by a dispatch.)
- Project pins: `.tebako-tools.yaml` at any directory — the dispatcher walks up from cwd; nearest wins.
- Extension slices (schema_minor 10): per-tool keys in the same two documents (project file wins over user config, as with versions):

FIGURE 1

```
metanorma:
  version: "1.16"                  # the existing chain,
unchanged
  slices: [metanorma-bsi@1.2.0]    # explicit pins: fetch-
on-miss, a
                                   # constraint mismatch
= SliceIncompatible
auto_slices: true                  # false kills the §2
step-3 store scan
```

One-off env: `TEBAKO_<TOOL>_SLICES=metanorma-bsi@1.2.0,…` (prepended to the pinned set). Pins override auto-discovery for the same slice name; a pin naming a slice installed at a DIFFERENT version attaches the pin exactly.

- Payload-level pins (store-config schema_minor 5, tebako#667): a `defaults:` KEY of the form `<registry>/<payload>` pins every entrypoint the payload declares — the one-line form of "this machine runs metanorma 1.16.9" for a 31-entrypoint suite:

FIGURE 2

```
defaults:
  "metanorma/metanorma": {version: 1.16.9}   # all 31
entrypoints
  metanorma-iso: {version: 1.16.8}           # specific
beats general
```

Version selection is a property of the payload, not of the command name; the payload pin sits in the user-default tier (env and project pins — per-tool by design — still beat it, as does a command-level `defaults:` entry for its own command). The registry alias rides the KEY (the value's `registry:` field would duplicate it — a payload pin carrying

© 2026 The Tebako Project

one is the named grammar error); the value carries `version:` only (`slices:` is per-tool). A version-less payload pin only scopes the payload's registry-default lookup (link 4) to the key's alias — the same rule as the tool form's registry-only entry. Two payload pins naming the same payload with different versions are the named ambiguity error, never a silent winner; a `/`-carrying key outside the grammar is the named grammar error at dispatch. `tebako-shim doctor` checks payload pins for dangling payload/version references exactly like qualified tool pins.

5. Distribution forms (both produced by `tebako press`)

1. Standalone tpkg (always per-platform): self-contained executable for users WITHOUT tebako.
2. Registry payload (`.tfs` + registry metadata): for dispatcher users — ONE universal image when pure-language; per-triplet variants only for native-extension apps.

6. Retirement gate

When tebako-shim ships, mnenv retires; metanorma becomes the first dogfood consumer (heavy native-ext app proving the whole stack: press → signed `.tfs` per (version × ruby line) → registry → dispatcher).

7. Errors (named)

- No compatible cached/downloadable runtime → the spec 06 exit 69 shape.
- Native-ext payload on a wrong-ABI-line runtime → named compatibility error, never a segfault.
- Shim target payload missing/corrupt → named error pointing at `tebako doctor`.

© 2026 The Tebako Project

- More than one ENABLED claim for a command (2026-09-05 amendment) → EX_TEBAKO_MANIFEST, naming every claiming payload; the hint names the routing verbs: pin `<cmd>: <payload>@<version>` in `.tebako-tools.yaml`, or disable one claim (`tebako-shim disable <cmd> --of <payload>`).
- A payload-qualified chain value whose payload is not installed, or is installed but neither declares nor exposes the command → the NotAProvider named error (EX_TEBAKO_MANIFEST), naming the pin, the payload, and the chain link the pin came from.
- A payload-only pin (`payload@`, empty version) → the grammar error (EX_TEBAKO_MANIFEST, §0): route out with `tebako-shim disable <cmd> --of <payload>`, never with an incomplete pin.
- An unparseable chain value at ANY link (env, project file, config default) → the grammar error, naming the link and the value (spec 23 §14's env-parse rule, extended to every chain link) — never a silent skip to the next link.
- A command whose only claims ride platform-skipped edges (spec 03 §2.3, schema_minor 9 — the edge's `triplets:` list does not cover this host) → the named "not available on this platform" refusal (EX_TEBAKO_UNAVAILABLE, 69), naming the claiming payloads — never the generic no-provider error, never a silent fallback.
- Extension slices (schema_minor 10):
  - A CONFIG-PINNED slice whose `augments` constraint fails the resolved base, whose language/abi edge fails the resolved runtime, or whose named `extension_point` the base does not declare → `SliceIncompatible` (EX_TEBAKO_MANIFEST), naming the slice, the pin's source link, and the failed check. (Auto-discovered slices fail SOFT — loud skip + journal; only pins fail hard.)
  - `augments` on the root payload's own manifest, or a slice whose mount would nest under another slice → `NestedAugment` (EX_TEBAKO_MANIFEST). Slices attach to the root payload, full stop.
  - A slice fetch that misses offline (TEBAKO_OFFLINE=1, pin not in the store) → the spec 05 cache-or-error shape, naming the slice.

8. Native exec from inside an image (the whole-chain model, locked)

Running a NATIVE executable from a mounted payload raises two problems: the exec'd binary loses the VFS view (it sees the host), and its dynamic dependency chain must resolve. Old tebako never solved this (its memfs was ruby-only; metanorma-on-tebako required host-installed inkscape). The locked model is three tiers — interposition-first, never FUSE:

© 2026 The Tebako Project

1. Dynamic native tools (the mainline): the preload interposition shim — `libtfs-preload`, injected via `LD_PRELOAD` (ELF), `DYLD_INSERT_LIBRARIES` (Mach-O), or DLL injection (Windows), mapping the libc/dyld file-IO family (open/stat/opendir/pread/dlopen…) onto `tebako_fs_*`. The launcher seeds the mount table via env (`TEBAKO_TFS_MOUNTS=image[:slot]:mount,…` — grammar in spec 17 §2.1); the binary AND its whole dynamic chain see the mounted image — **no extraction, no chain problem**. retrace (in-family: linux/macOS/windows CI, v2 config-driven interception) is the reference technique. Bonus: interposed IO flows through the same `host_policy` — **jails extend to these binaries** (spec 08). Limit: dynamic binaries only; the shim is a TFS consumer, never a format. **SHIPPED (roadmap 30): `crates/libtfs-preload` + `tfs exec <image>[:mount] [--image …] [--jail <spec>] -- <cmd>` in tfs-cli — macOS and linux-gnu first-class, windows later. Coverage (roadmap 39):** interposed surface open/openat/creat(the tebako#444 audit's adjacent-write finding: creat(3) IS open(O_WRONLY|O_CREAT|O_TRUNC) spelled as its own libc symbol — glibc builds it over libc-internal open aliases and libSystem gives it its own stub, so un-interposed it bypassed the write gate everywhere; it routes exactly like a write-mode open: a memfs-held path is EROFS, a host path is policy-gated before the real creat runs)/stat/ lstat/fstat/fstatat(+fstatat64/statx/_xstat/_lxstat/*fxstat/* fxstatat and the LFS open64/stat64/lstat64/fstat64/pread64/lseek64 family and the pre-2.33-glibc versioned _xstat64/_lxstat64/ *fxstat64 forms and the _FORTIFY_SOURCE* read_chk wrapper on linux)/access/faccessat/opendir/readdir(+readdir64)/ readdir_r/ rewinddir/telldir/seekdir/dirfd/closedir/pread/read/lseek/ mmap(memfs fd → private anonymous mapping pre-filled from the VFS, +mmap64 on linux)/ close(+the plain and $NOCANCEL spellings on x86_64 darwin — the libc crate maps `libc::close` to `close$NOCANCEL` there, C binaries (the JVM's libjava/libjli/libzip) import plain `close`; both route to the shim)/dup/dup2(tebako#534 — libxml2's xmlInputFromFd calls dup(fd) unconditionally, so every VFS fd died EBADF there: the clone lands on a shim-managed flagged fd carrying the SAME open-file description — the offset is shared, per POSIX; dup and fcntl(F_DUPFD) leave FD_CLOEXEC off the clone, F_DUPFD_CLOEXEC sets it; dup2 atomically closes a live memfs target and rebinds the number to the source's description (the target resolves to the same VFS file), and dup2(fd, fd) is the liveness-checked no-op; a memfs source with a HOST-numbered dup2 target is the named error ENOTSUP — the fd routing keys on the flag bit, so a host number can never name a memfs description — never EBADF-by-default; a dead memfs fd is EBADF, the kernel's own answer; host-numbered dup/dup2 sources pass through untouched)/fcntl(the descriptor commands on a memfs fd are served from the shim's own fd table — open/openat seed the close-on-exec bit from O_CLOEXEC, F_GETFD/F_SETFD read/write it, F_GETFL answers the truthful read-only status word, F_SETFL is an accepted no-op, a flagged-but-closed fd is EBADF, the dup-class commands F_DUPFD/F_DUPFD_CLOEXEC clone the open-file description exactly like the dup/dup2 shims above, host fds pass through untouched — CPython's io.FileIO boot path fcntls every fresh source fd with raise=1, so unshimmed the interpreter died importing `encodings` before any user code)/fopen(tebako#444: read-only modes (r with no +) materialize the memfs original through dlmap2file exactly like dlopen — stdio's internal open path never touches the interposed open; the write-ish modes (w/a bases, every
   — update form incl. r+, the x-forms) ride the SAME spec 08 write

© 2026 The Tebako Project

gate as open(O_WRONLY|O_CREAT|O_RDWR) BEFORE the real fopen touches the host — a memfs-held path is EROFS, a jailed write outside an rw grant is EPERM by name; pre-fix the write modes sailed through ungated and r+ was misclassified as a read)/mkdir/unlink/rename + dlopen + execve/posix_spawn/posix_spawnp

- the realpath family (realpath, plus macOS's realpath$DARWIN_EXTSN; canonicalize_file_name and the fortified __realpath_chk on linux): glibc's realpath(3) walks the path with libc-INTERNAL stat/readlink aliases no PLT interpose sees, so a covered path MUST be answered by the shim with the MOUNT spelling — the in-image link hops resolved (spec 11 §9's rule: ELOOP past 32, a dangling link ENOENT) and the result lexically normalized. The host resolver's canonicalization must never rewrite a VFS path (the 2026-09-03 dogfood linux-gnu jing ClassNotFound: usrmerge /lib→usr/lib leaked into the JDK's URLClassPath on a root-mounted payload, and the jar lookup silently dropped a byte-perfect jar); covered-but-not-held paths still pass to the host realpath (the /etc/localtime discipline); `TEBAKO_JAIL` carries the spec 08 §1 env form (`open|deny` + `host:mount:ro|rw` grants + `@` argument files); the ENTRYPOINT (when in-image) is materialized through `dlmap2file` (execve needs a host path — one copy per exec, gc later), and execve/posix_spawn of MEMFS paths materialize the same way (a tool spawning an in-image helper needs no extraction; environ propagates, so grandchildren stay in the VFS); every *at shim gates its fd branch on `dirfd >= 0` (AT_FDCWD carries the fd-flag bit — the runtime-builds regression class, pinned by tests); the mmap shim gates on `fd >= 0` and `!MAP_ANONYMOUS` (fd -1 carries the same bit — the JVM PaX-check regression class); exec of a HOST binary is not policy-gated (not an IO route in the policy's op classes — the child's own IO stays jailed via env propagation); SIP platform binaries strip `DYLD_*` and leave it; a mount at `/` is legitimate under the union mount model (spec 03 §6 / spec 17 §1 — the app payload mounts there): covered-but-not-held paths fall through to the host with the jail consulted; fork is not interposed but its child side is GUARDED — a `pthread_atfork` child handler arms a process-global flag and every shim's engine entry passes through to the real libc while it is set (the backends are not fork-safe: dwarfs-t's block-cache worker pool dies at fork, so a backend-touching call in the child — e.g. the execve materialization probe under a `/` mount — would wait on threads that no longer exist; the 2026-08-22 git-clone deadlock, runtime 0.16.4); glibc runs NO atfork handlers for vfork(2), and a vfork child shares the parent's whole address space with the parent thread suspended in the kernel, so the guard carries a PID backstop — the constructor records the initializing pid, and an engine entry under a different pid is a fork/vfork child in its pre-exec window and passes through exactly like the atfork flag's child (the 2026-09-03 dash vfork deadlock in the dogfood repro; exec preserves the pid, so the exec'd image's constructor re-stores the same value and the gate re-opens at exec); and a fork child that execs re-enters a fresh shim through the inherited preload env, so the grandchild rule above is unaffected; not interposed: fork, dup3 (linux-only, no tier-1 consumer — its flags argument would need the cloexec plumbing the POSIX dup class already has), `openat2` (glibc exposes no wrapper or symbol on linux-gnu — nothing to interpose; a raw `syscall(2)` caller bypasses userland interposition by construction), fstatat64 on macOS (the legacy 32-bit-inode layout), `__fxstatat64` and the write-side pwrite64/ftruncate64/statvfs64 family plus readv/preadv/sendfile/copy_file_range on memfs

© 2026 The Tebako Project

fds on linux, fdopendir (memfs directories are never fd-opened), the internal-walk family glob/nftw/ftw/wordexp (UNAUDITED — their path walks ride libc-internal aliases, the same bypass class as realpath; no importer observed on the JDK/ruby boot paths, so this is pinned debt, never silently assumed covered), and syscall()-direct IO (raw syscalls bypass userland interposition by construction); `dirfd` of a memfs stream answers -1/ENOTSUP; `getdents64` on a memfs fd answers ENOTDIR (VFS directories enumerate via opendir, never fds).

2. Static/self-contained tools: tebako is the EXECUTOR, not an injector — it spawns and owns every child, so mediating a child's syscalls is the execution contract itself, not hijacking (a side-loader would need to BECOME the executor or the kernel to hold the same position — outside the threat model by definition). As the executor, tebako mediates statics two ways: (a) Kernel-mediated VFS (Linux, tier 2a): `SECCOMP_RET_USER_NOTIF` — the executor installs a seccomp filter at spawn; the kernel forwards the child's file syscalls (openat/statx/...) to the executor, which serves them from the mounted TFS images and enforces the same `host_policy` (spec 08) — full VFS + jails for static binaries, no extraction, child unaware. Cost: per-syscall user/kernel round trips (io-heavy tools notice); platform: Linux only (the API does not exist elsewhere). (b) Extract-to-exec-cache (everywhere, tier 2b): `~/.tebako/exec/<sha256>/...`, 0755, verified at install — the manifest-declared closure, content-addressed, shared. On macOS/Windows (no executor-side syscall mediation API) this is the static path; such binaries see only the host FS (no VFS, no jails — a platform-API statement, not a trust statement). Nuances: a static binary BUILT against the `tebako_fs_*` ABI carries the VFS inside itself (the "libtfs for everyone" embedding story — build-time property, not imposable on an arbitrary binary). Prefer tier 1 (dynamic) for io-heavy tools; tier 2a covers the rest on Linux.
3. Interpreted payloads (ruby): in-process io-routing patches virtualize file IO — the existing mechanism, no exec at all.

   — FUSE is NOT an exec mechanism. It remains an optional, host-dependent HUMAN convenience (`tfs mount`, spec 11 access matrix #2) — never on the package/exec path. (Correction 2026-07-26: tebako has never used FUSE; interposition is the tebako way.)

   — ptrace/syscall-tracing (retrace-v1-style tracing for vulnerability research) is a debugging tool, not an exec foundation.

8a. The virtual cwd (tebako#734)

Decision: virtual-cwd, named-limitation floor. The process's cwd is tracked as a VFS-aware value: `chdir(2)` into a mounted path SUCCEEDS (the preload interposes it and records the VFS point); every relative-path interposition resolves against the tracked cwd — a VFS cwd makes relative paths route into the image, a host cwd keeps today's behavior. The floor until each surface lands: a surface that cannot yet honor a VFS cwd fails with a

© 2026 The Tebako Project

NAMED EXDEV-class error naming the cwd ("cwd is a VFS path this operation does not serve"), never a bare ENOENT that reads as image corruption.

- `press -c/--cwd`: the generated stub may `Dir.chdir` into the VFS (the feature was broken-broken before — bare ENOENT on its own stub); the stub's chdir now succeeds and the interpreter's relative loads resolve in-image.
- The pre-existing assumption retires (`route.rs`: "the cwd is always host"): every `current_dir()`-based relative resolution in the preload joins the VIRTUAL cwd when one is set.
- Bundler's git source (the #119 reporter): `SharedHelpers.chdir` into the checkout now works in-image.
- Host escapes: `chdir("..")` past the VFS root reaches the host parent exactly as a path op would (the jail policy governs); no new wall is invented.

9. Environment composition (locked 2026-07-27)

Env across the mount stack (runtime + toolkit layers + data layers + app) composes by RULE, never by accident:

- Path-class vars (GEM_PATH, PATH-in-image, TEBAKO_LD_PATH): app payload first, then toolkit layers in `requires` order, then the runtime — earlier wins. The runtime's own defaults come LAST (it is the fallback, not the override).
- Scalar vars: the app's manifest wins over toolkit, toolkit over runtime; the dispatcher's computed values (TEBAKO_RUNTIME_IMAGE, TEBAKO_TFS_MOUNTS, jail env) always win over every manifest.
- Host env scrub: on handoff the dispatcher removes the variables the press scrubbed (RUBYOPT/RUBYLIB/BUNDLE_* — the M7 scrub list) so the host environment never leaks semantics into the payload; other host vars pass through unless the jail says otherwise.
- Per-entrypoint overrides (suite entries, spec 03 §6) apply last for that entry only.

9.1 The interp_env chain (IMPLEMENTED 2026-10-07 — tebako#559; spec 03 §2.7)

Interpreter-option defaults compose by their own chain, parallel to the rules above and NEVER through the M7 scrub (interp_env keys are interpreter options, not rubygems

© 2026 The Tebako Project

semantics — they are never scrubbed). Per dispatched entry, per declared key, FIRST SET wins, highest layer first:

1. User process env — the operator's invocation environment. A key already set there is the user's explicit choice: it always wins and is never rewritten.
2. User config — `~/.tebako/config.yaml`'s per-tool `env:` map (spec 07 §4; store-config schema minor 3): the operator's standing local policy for one tool.
3. Package manifest — `entries[].interp_env` (spec 03 §6), the standalone/package form only: the packager's press-time refinement (already composed with the slice's L1 declarations by the press). Managed dispatch has no L2 — this layer is absent there by construction.
4. Payload manifest — the slice author's L1 `interp_env:` (spec 03 §2.7), read from the store's manifest mirror in managed dispatch.
5. Interpreter built-in — what the interpreter does when nothing declares the key (the declaration is a default, never a requirement).

The dispatcher computes the effective map at dispatch and exports each winning key that is NOT already in the process env (layer 1's win is passive: the variable is simply there). No key is ever scrubbed, rewritten, or guessed; an unset-declared key stays unset (never a synthesized "0"). The effective map and each key's provenance layer are reportable through the spec 15 §4 surface — resolution is auditable, never silent. The standalone bootstrap reads no store config — the managed shim owns that tier — so a stitched package's chain is process env over the package manifest (layers 1 and 3; layers 2 and 4 are absent by construction).

10. The runtime-own command surface (DRAFT — the user-direct grammar)

A runtime's own tools (the interpreter itself, its package manager, its loaders) are first-class commands, not only what a payload's `expose:` edge re-exports. The factory's env image declares the runtime's entrypoints (the additive entrypoints list — spec 30 §2; the node factory declares node/npm/npx/corepack; the bun factory bun/bunx; the openjdk factory java/keytool/...); the user reaches them directly:

tebako run <engine>[:<command>] [<args>...]

- The bare form (`tebako run node`) runs the runtime's DEFAULT entry (the interpreter — the env image's `layout.interpreter`), with the args handed verbatim.

© 2026 The Tebako Project

- The command form (`tebako run node:npm`) resolves the named entrypoint in the runtime image's declared entrypoints — an undeclared name is a named error listing the declared set, never a guess.

The `run` target grammar (MECE, locked). `tebako run` has exactly two target classes, decided on the first positional token, in this order:

1. **A token naming an existing file** is a package — the §2/pkg surface of this verb, unchanged. (A windows drive path always lands here while the file exists; spell the engine form for the engine.)
2. **Otherwise the token parses as `<engine>[:<command>]`** — the engine grammar: `<engine>` is the runtime payload's `engine` key (spec 04 §2 MINOR 1 — bare lowercase name), `<command>` one of the resolved runtime's declared entrypoints. A token that is neither an existing file nor a well-formed engine reference is the §7 usage error, naming the token.

A token is never both: the file reading is total (a file either exists or not), the engine reading applies only when no file answers. A bare engine name shadowed by a same-named file in the cwd is reached by the command form (`tebako run node:node`) — deterministic, journaled, never a silent pick.

Resolution: the engine resolves against the store FIRST (the newest compatible cached runtime), then the registries (download, verify, cache — the same fetch path as every payload's runtime requirement). The version chain per engine: `TEBAKO_<ENGINE>_VERSION` env → the nearest `.tebako-tools.yaml` → `~/.tebako/config.yaml`'s `defaults:` → the registry's newest. All local surfaces, never a registry-side route (the provider-routing rule above). The engine name matches a registry entry's `engine` key, never a display name; no configured registry carrying the engine is the §7 no-such-engine error.

Dispatch: the handoff is the spec-17 wire exactly — the wrapper exe with `TEBAKO_RUNTIME_IMAGE=<env image>` + `--tebako-entry <command>` (or the bare form's default) + the user args; the driver composes `[interpreter, <the entrypoint's args_default…>, <entry>, <args>…]` per spec 29 §1. This surface parses NO flags: every token after the engine reference is the engine's own argument, verbatim (a `--jail` there reaches the engine, never tebako). The jail posture is the user's own policy riding `TEBAKO_JAIL` in the inherited environment (the driver enforces it per spec 17), open by default — a runtime-own command is the user's own process, not a payload's.

The shim surface: `tebako install <engine>[:<command>]` registers shims/<command> (the default entry's name for the bare form — node, bun, `java`) dispatching argv0 → the engine's runtime resolution → the same spec-17 wire. The install target's engine reading WINS a bare engine-shaped token it answers (a cached runtime of the engine, a config `runtimes:` preference, a registered registry's `kind: runtime` entry carrying the engine, or the ruby default); a same-named payload stays reachable through its qualified forms (`<alias>/<name>`, the reference forms). **Collision rule (MECE):** a PAYLOAD-provided name wins over a runtime-own name of the same spelling (the payload declared the tool; the runtime surface is the fallback when no payload provides it) — the provider-routing surfaces of §2 answer collisions locally, never by registry. A runtime-

© 2026 The Tebako Project

own shim records its engine class in the shim metadata (`shims/.runtime-own.yaml`, command → `{engine, entry}` — the entry is the wire token: `self` for the bare form, the command name otherwise), so `tebako shim list/disable` answer the class explicitly and a later payload install claiming the name displaces the runtime-own claim with a loud notice (never a silent shadow). The §0 disable selectors gate a runtime-own claim through the engine riding the payload slot: `all` and `<engine>@all` refuse the dispatch by name; a bare version selector has no axis to gate (the engine's version chain owns versions).

The presses stay honest: a runtime-own shim name never ships a binary — the shim is the link; a press that bundles its own tool keeps the payload-provided name (the payload wins the name by the collision rule above).

Errors (named, §7's table extended):

- No configured registry carries the engine, and the store holds no runtime declaring it → `EX_TEBAKO_UNAVAILABLE` (69), naming the engine and every registry consulted — the no-default-service rule makes "unknown engine" and "known but unreachable" the same shape.
- The command form naming an entrypoint the resolved runtime's env image does not declare → `EX_TEBAKO_MANIFEST`, naming the engine, the asked command, and the declared set.
- `TEBAKO_OFFLINE=1` with no compatible cached runtime → the spec 05 cache-or-error shape, naming the engine.
- A malformed engine reference (`node:`, `:npm`, an engine name outside the name grammar) → the §7 usage error, naming the token.

© 2026 The Tebako Project